OPERATING SYSTEM

UNIT 4
CONCURRENCY

Prof. N. K. PATIL
Department of Computer Engineering
Sinhgad Institute of Technology, Lonavala

Email nkp.sit@sinhgad.edu
Contact No.+91-9423340927

sppu_te_be_comp

Study material provided by: Vishwajeet Londhe

Join Community by clicking below links

Telegram Channel

https://t.me/SPPU_TE_BE_COMP

(for all engineering Resources)

SPPU Engineering & Technical UPDATES

WhatsApp channel

WhatsApp Channel

(for all Engg & tech updates)

https://whatsapp.com/channel/0029ValjFriICVfpcV9HFc3b

@SPPU_ENGINEERING_UPDATE

Insta Page

(for all Engg & tech updates)

https://www.instagram.com/sppu_engineering_update

sppu_te_be_comp

CONTENTS

1. Concurrency, persistence, shared data
2. Uncontrolled scheduling, The wish for atomicity
3. Thread api: need of threads, thread creation, thread completion
4. Locks: the basic idea, pthread locks, building a lock, evaluating locks
5. Controlling interrupts, failed attempt, just using
G. Semapohres: definition, binary semaphores (locks), semaphores for ordering
7. The producer/consumer (bounded buffer) problem, reader-writer locks
8. Dining philosophers' problem, How to implement semaphores
5. Common concurrency problems.

sppu_tt_bo_comp

COURSE OUTCOME

- CO4: **Describe** the working of concurrency and locking mechanism in operating systems.

sppu_te_be_comp

Web browser

A modern web browser is a practical, everyday example of concurrency that students use all the time.

- **The process:** A web browser application running on your computer is a process.
- **The threads:** Within that single browser process, multiple threads of execution operate concurrently.
  - **UI Thread:** Handles your interaction with the browser window, like clicking buttons, scrolling, or opening menus.
  - **Multiple Tab Threads:** Each individual tab can run as its own thread. This is why if one tab crashes, the others usually remain unaffected.
  - **Network Request Thread:** Fetches new data from a server, like an image or video file.
- **The scenario:** Imagine you are scrolling through a webpage (UI thread) while an animated advertisement is loading (network request thread) and music from another tab is playing in the background (another tab thread). All of these activities appear to be happening at the same time, even on a computer with a single-core processor. The operating system's scheduler rapidly switches between these threads to give the user the illusion of simultaneous execution.

supu_te_be_comp

Multiplayer video game

Gaming is a high-performance scenario where concurrency is essential for a smooth user experience.

- **The threads:** A modern video game uses multiple threads for different, simultaneous tasks.
  - **Rendering Thread:** Manages the game's graphics and updates what appears on the screen.
  - **AI Thread:** Controls the behavior of non-player characters (NPCs).
  - **Audio Thread:** Handles music and sound effects.
  - **Network Thread:** Manages communication with other players' computers during online multiplayer gameplay.
- **The scenario:** While playing a game, you can see a complex, animated world (rendering), hear music and sound effects (audio), and interact with other players in real-time (network), all without a noticeable delay. The OS and game engine work concurrently to keep the game responsive and fast.

spu_te_be_comp

WHAT IS CONCURRENCY?

- Ability to handle multiple tasks in overlapping time periods
- Achieved via context switching (single CPU) or parallelism (multi-core CPUs)
- Concurrency is essential in modern operating systems due to the increasing demand for multitasking, responsiveness, and resource optimization.
- Applications include web servers, databases, scientific simulations, and multimedia processing.

sppu_te_be_comp

How Concurrency Works?

- Single-core CPU: Rapid context switching
- Multi-core CPU: True parallel execution
- Enhances system throughput and responsiveness

sppu_te_be_comp

Principles of Concurrency

- Shared resources
- Execution time depends on:
  - Other processes
  - Interrupts
  - Scheduling

Note: "Concurrency is not just about speed—it's about coordination."

sppu_te_be_comp

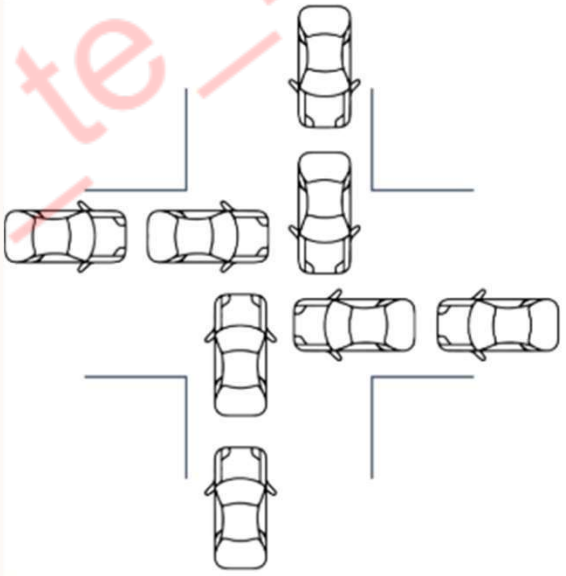
Why Allow Concurrent Execution?

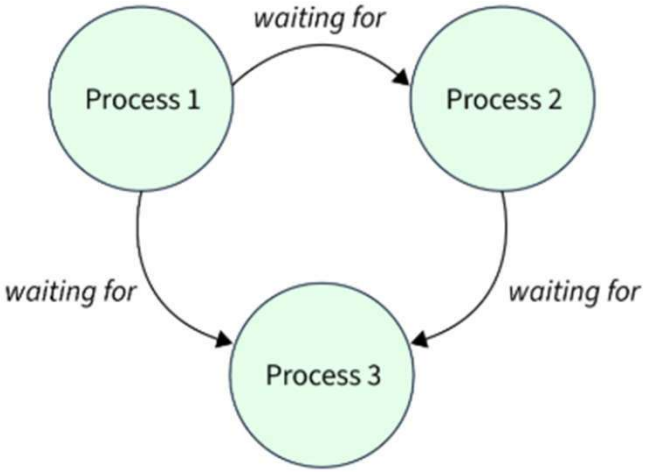
- Resource Utilization
- Responsiveness
- Throughput
- Real-time Processing

sppu_te_be_comp

Problems in Concurrency

- Race Condition
- Deadlocks
- Priority Inversion
- Resource Starvation





sppu_te_be_comp

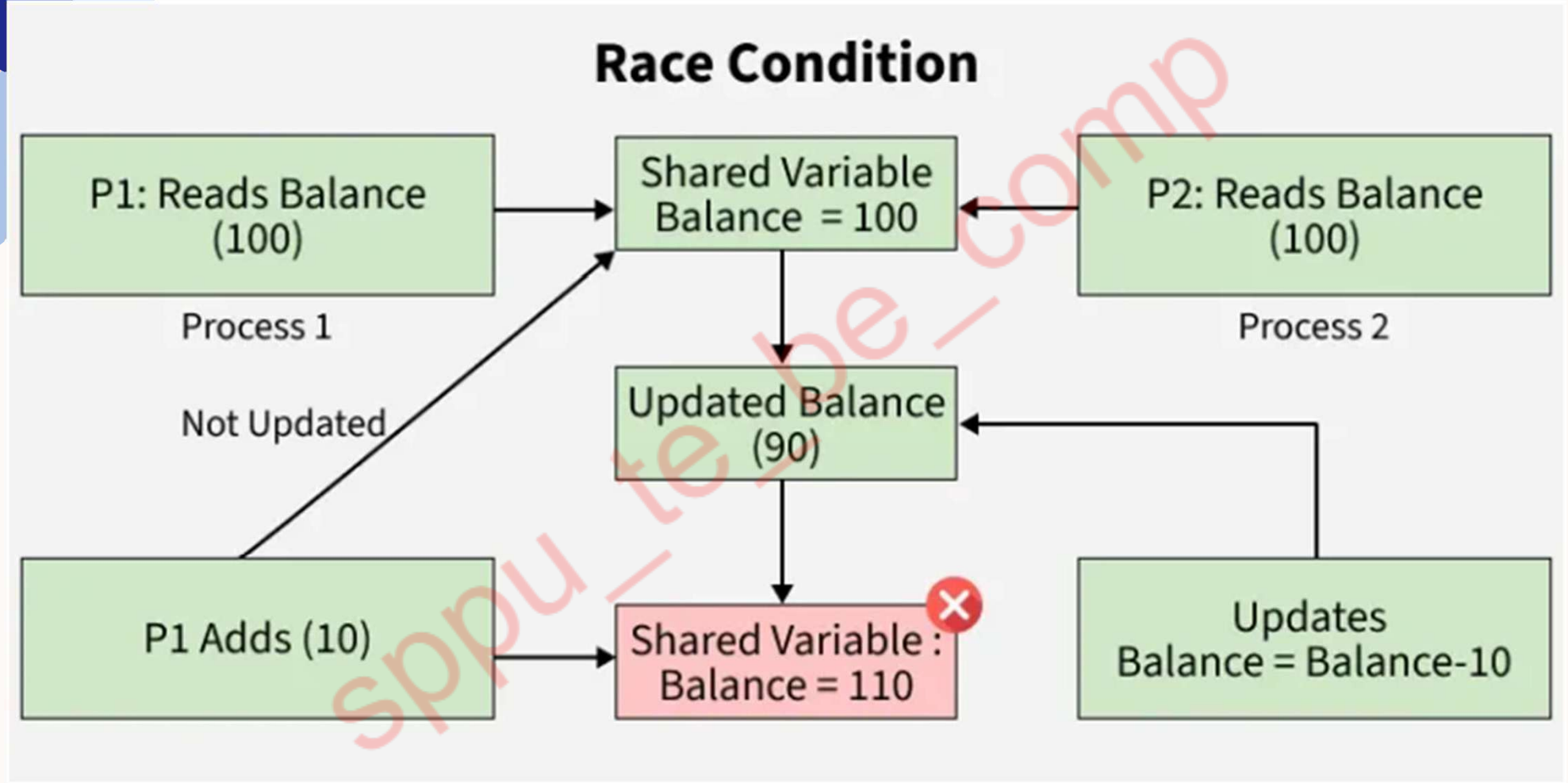
EXAMPLE: TWO PROCESSES UPDATING A SHARED VARIABLE

Let's take a shared variable balance = 100 and two processes P1 and P2:

- **P1 wants to add 10 to balance.**
- **P2 wants to subtract 10 from balance.**

Scenario without synchronization:

sppu_te_be_comp



Race Condition

- P1 reads balance = 100 and prepares to add 10.
- Before P1 updates the balance with the new value (110), it is interrupted by the process P2.
- P2, unaware of P1's action (of adding 10), reads the balance as 100 (incorrect) and prepares to subtract 10.
- After subtracting, P2 updates the balance to 50 and then P1 resumes and writes the balance as 110 which is incorrect now.

In many cases, the final balance may incorrectly be 110 or 50, instead of the expected 100. This is a classic race condition.

Spcu_to_be_comp

RACE CONDITIONS

- **Situations like this where processes access the same data concurrently and the outcome of execution depends on the particular order in which the access takes place is called a race condition**
- **How must the processes coordinate (or synchronise) in order to guard against race conditions?**

sppu_te_be_comp

Advantages of Concurrency:

- Improved Performance
- Responsiveness
- Scalability
- Fault tolerance

sppu_te_be_comp

Disadvantages of Concurrency:

- Complexity
- Overhead
- Race Conditions
- Resource Management

sppu_te_be_comp

CONCURRENCY: PERSISTENCE

- **Persistence in concurrency refers to the ability for data and the state of concurrent processes to survive the termination or failure of the process that created them.**

- **It ensures that the results of these operations are not lost when a program or the system shuts down unexpectedly.**

sppu_te_be_comp

- Key aspects include:
  - State Survival
  - Non-volatile storage
  - Operating system role
  - Concurrency challenges
- Example
  - Word processors
  - Picture editors
  - etc

spnu_te_be_comp

CONCURRENCY: SHARED DATA

- When multiple concurrent processes access this shared data without proper synchronization, it can lead to race conditions, where the final result depends on the unpredictable order of execution, potentially causing data corruption and system instability.
- Examples of Shared Data includes shared memory, shared variables, files, system resources.

sppu_to_be_comp

How to Manage Shared Data

- Mutual Exclusion (mutexes)
- Semaphores
- Atomic Operations

sppu_te_be_comp

MULTICORE PROGRAMMING

Multicore or multiprocessor systems putting pressure on programmers, challenges include:

- **Dividing activities**
- **Balance**
- **Data splitting**
- **Data dependency**
- **Testing and debugging**

Parallelism implies a system can perform more than one task simultaneously

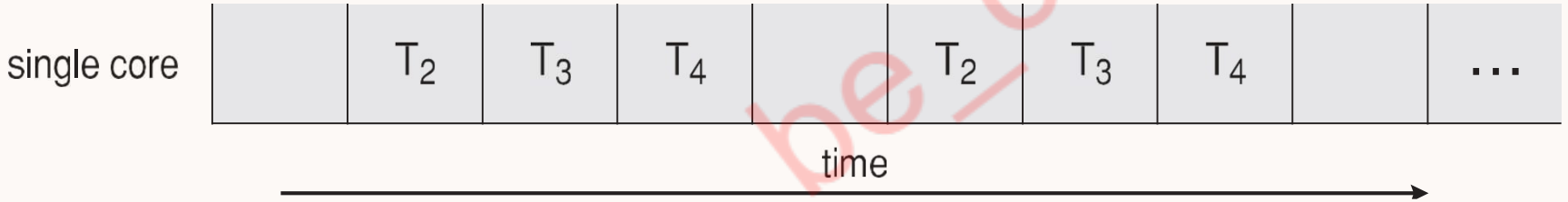
Concurrency supports more than one task making progress

- **Single processor / core, scheduler providing concurrency**

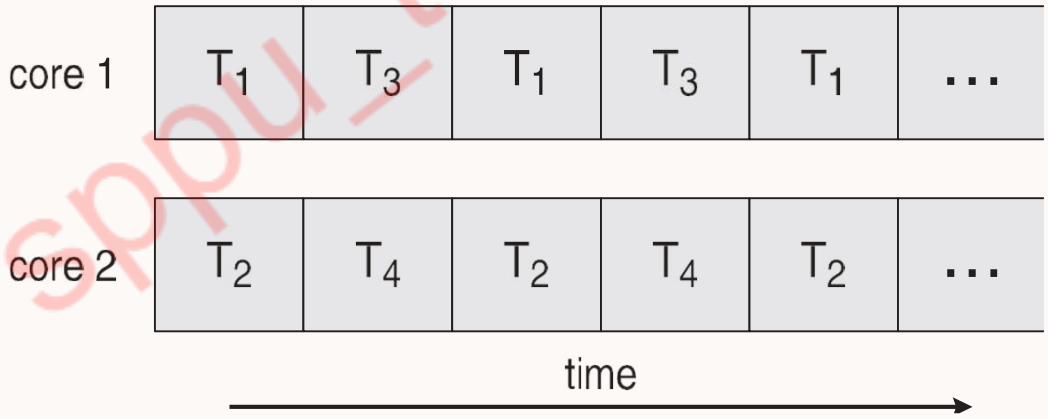
sppu_te_be_comp

CONCURRENCY VS. PARALLELISM

- **Concurrent execution on single-core system:**



- **Parallelism on a multi-core system:**



sppu_to_be_comp

THREADS API

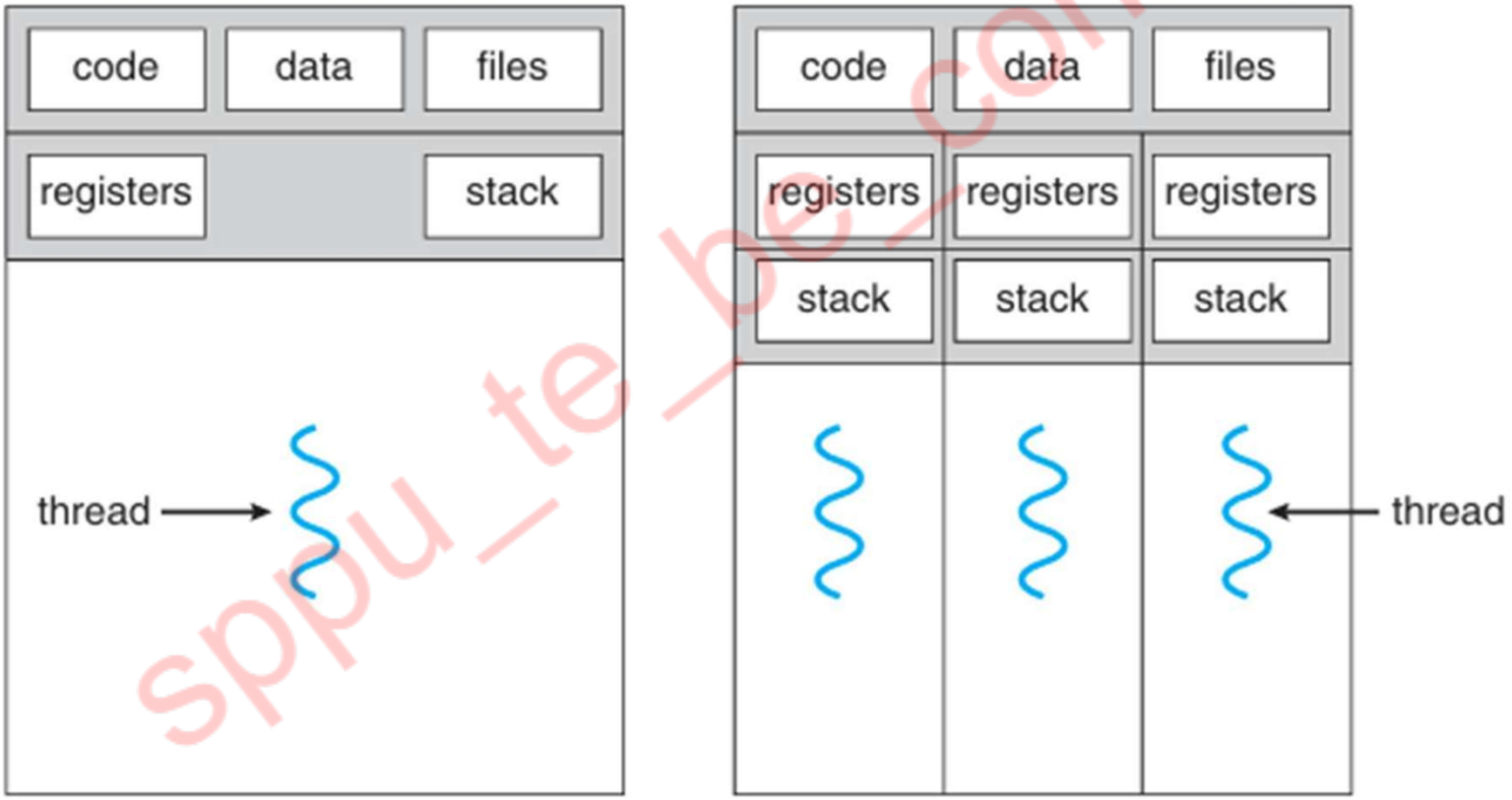
- Thread is a single sequential stream inside a process.
- Threads have same properties same like process that is why they are also known as light weight processes.
- Every thread has a place with precisely one process and no thread can exist outside a process.
- Every thread has a different flow of control.
- Threads have been effectively utilized in implementing network servers and web server.
-

sppu_te_be_comp

- Most modern applications are multithreaded
- Threads run within application
- Multiple tasks with the application can be implemented by separate threads
  - Update display
  - Fetch data
  - Spell checking
  - Answer a network request
- Process creation is heavy-weight while thread creation is light-weight
- Can simplify code, increase efficiency
- Kernels are generally multithreaded

sppu_te_be_comp

For example, in a web browser, one thread might handle the user interface while another downloads a file.



sppu_te_be_comp

BENEFITS OF THREAD

- **Responsiveness–** may allow continued execution if part of process is blocked, especially important for user interfaces
- **Resource Sharing –** threads share resources of process, easier than shared memory or message passing
- **Economy –** cheaper than process creation, thread switching lower overhead than context switching
- **Scalability –** process can take advantage of multiprocessor architectures

sppu_te_be_comp

THREAD FUNCTIONALITY

1. **Thread States – key states are running, ready and blocked. Four basic operation are :**
   i. **Spawn**
   ii. **Block**
   iii. **Unblock**
   iv. **Finish**

2. **Thread Synchronization – All the thread of a process share the same address space.**

sppu_te_be_comp

TYPES OF THREADS

In an operating system, threads are broadly classified into two main types:

- **User-Level Threads (ULTs) and**
- **Kernel-Level Threads (KLTs).**

Apart from these two, there is one more types of thread i.e. Hybrid or Combined.

spnu_te_be_comp

1. **User-Level Threads (ULTs) –**

- **Managed by:** User-space libraries, runtime systems, or programming languages.
- **Kernel Awareness:** Kernel is unaware of user-level threads and treats the entire process as a single-threaded entity.
- **Advantages:**
  - **Fast**
  - **Flexible**
- **Disadvantages:**
  - **Blocking Issue**
  - **No Multi-Core Support**

sppu_te_be_comp

2. Kernel-Level Threads (KLTs) -

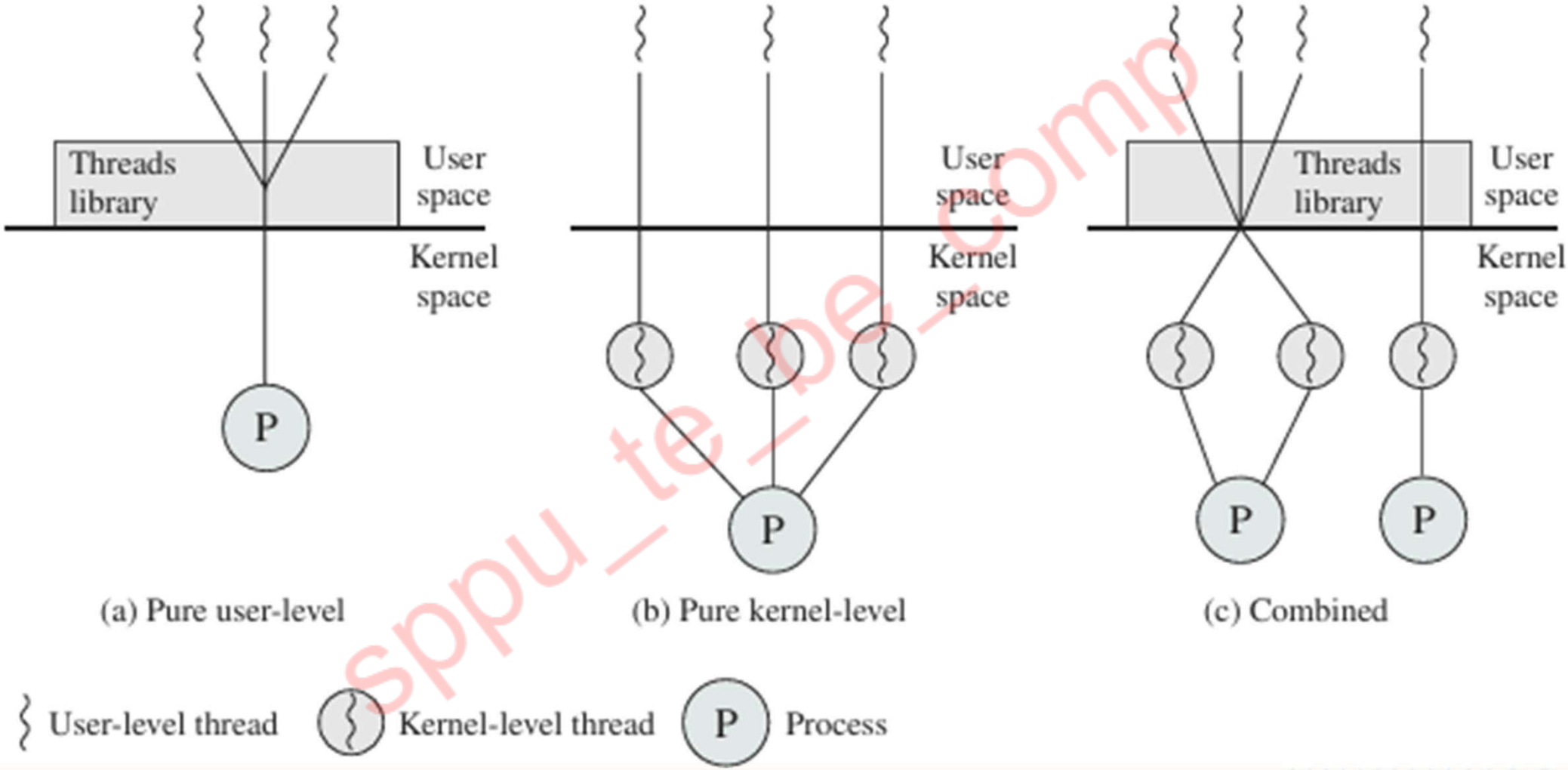
- **Managed by:** The operating system's kernel.
- **Kernel Awareness:** The kernel recognizes each kernel-level thread and can schedule them individually.
- **Advantages:**
  - **Multi-Core Utilization**
  - **Better Blocking Handling**
- **Disadvantages:**
  - **Higher Overhead**
  - **Slower**

sppu_te_be_comp

3. Hybrid Threads –

- Some systems also use hybrid models that combine both approaches, aiming to leverage the benefits of user-level threads' efficiency and kernel-level threads' parallelism and control.

sppu_te_be_comp

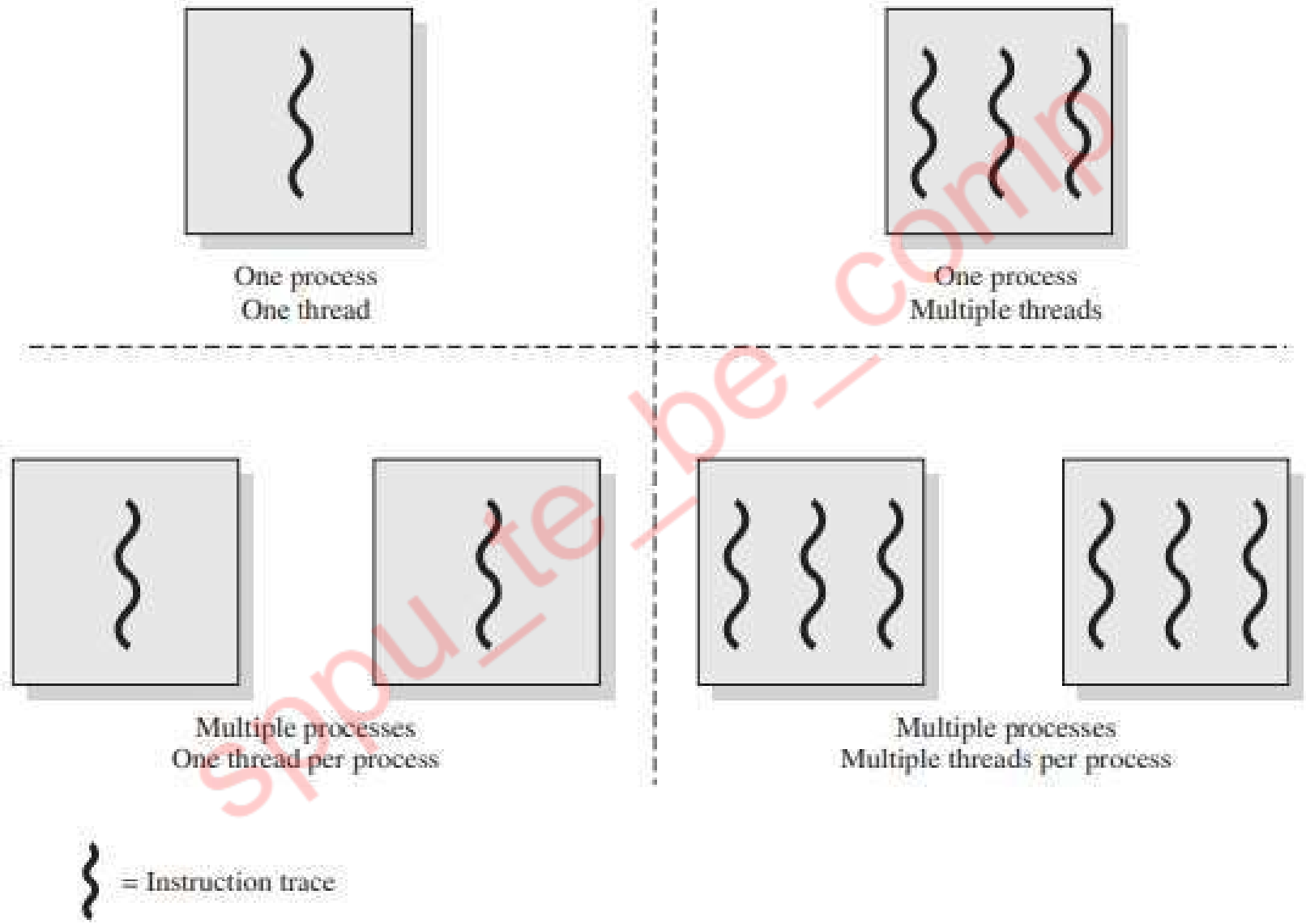


sppu_te_be_comp

MULTITHREADING

- It refers to the ability of an OS to support multiple, concurrent paths of execution within a single process.
- The traditional approach of a single thread of execution per process, in which the concept of a thread is not recognized, is referred to as a single-threaded approach.
- A Java run-time environment is an example of a system of one process with multiple threads.

sppu_te_be_comp

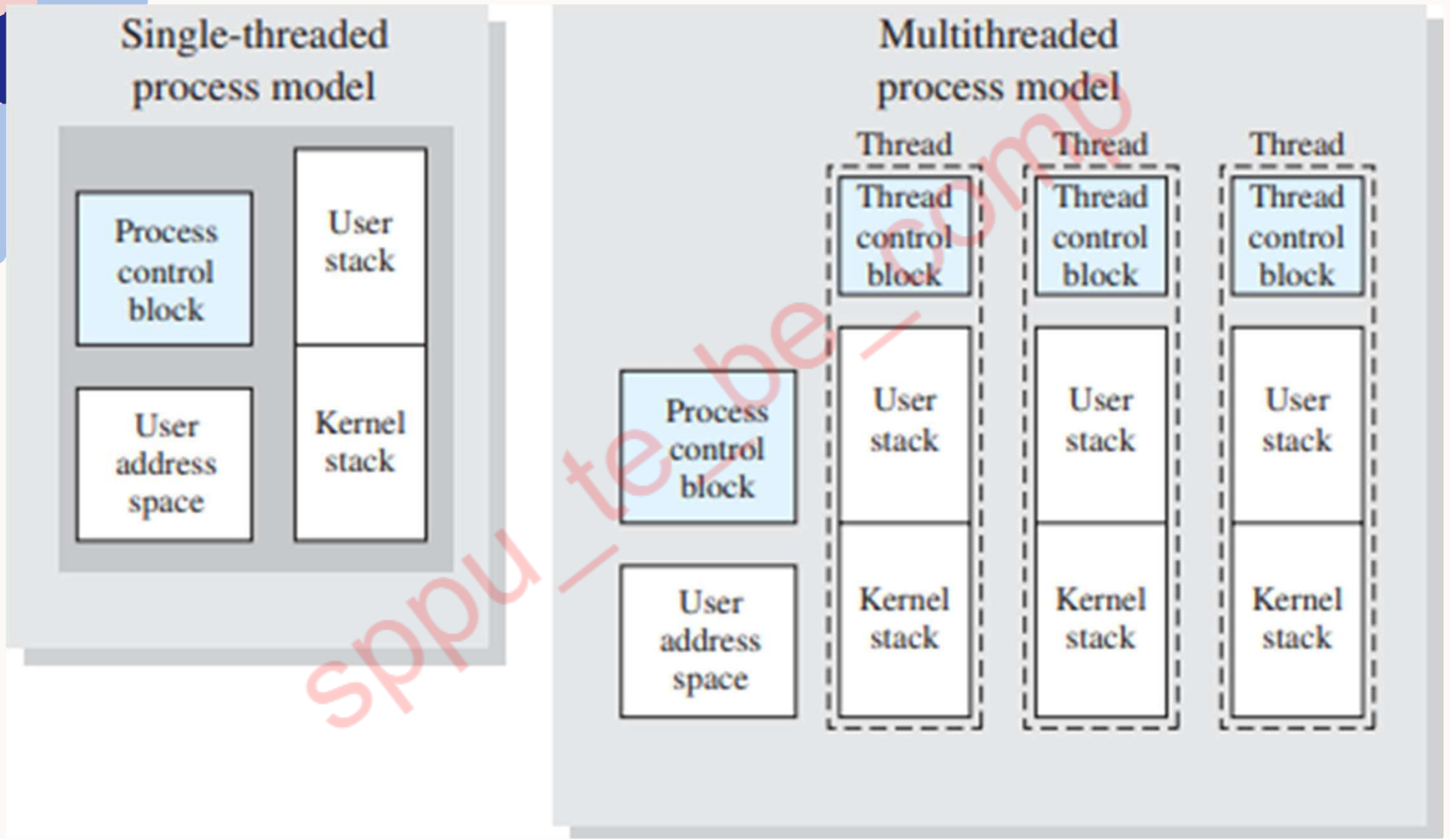


sput_te_be_comp

Within a process, there may be one or more threads, each with the following:

- A thread execution state (Running, Ready etc.)
- A saved thread context when not running; one way to view a thread is as an independent program counter operating within a process.
- An execution stack
- Some per-thread static storage for local variables.
- Access to the memory and resources of its process, shared with all other threads in that process.

spu_te_be_comp



sppu_te_be_comp

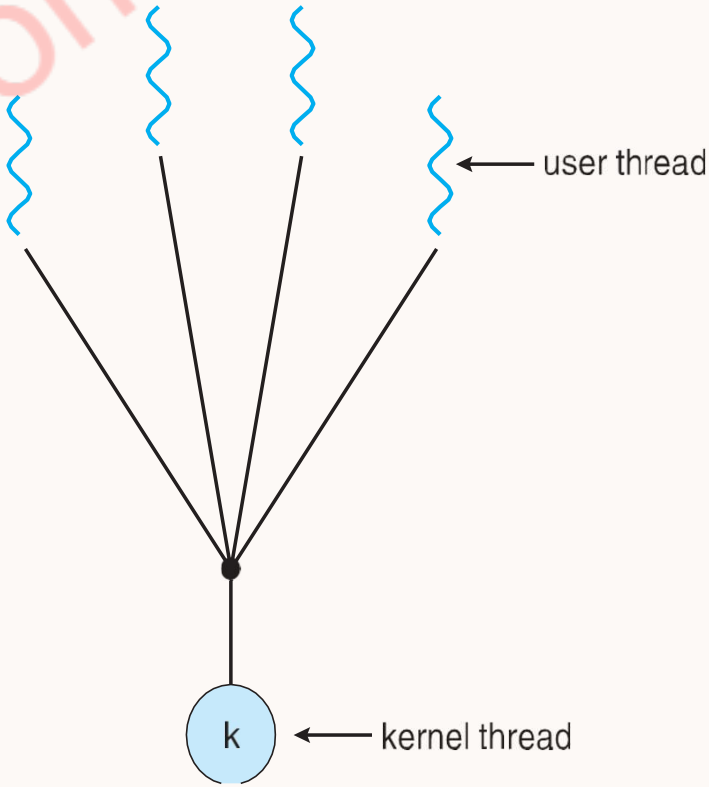
MULTITHREADING MODELS

- User threads are supported above the kernel and are managed without kernel support, whereas kernel threads are supported and managed directly by the operating system.
- Windows, Linux, Mac OS X, and Solaris— support kernel threads.
- 3 common ways of establishing a relationship between user level thread and kernel level thread: the many-to-one model, the one-to-one model, and the many-to many models.

sppu_te_be_comp

1. MANY-TO-ONE

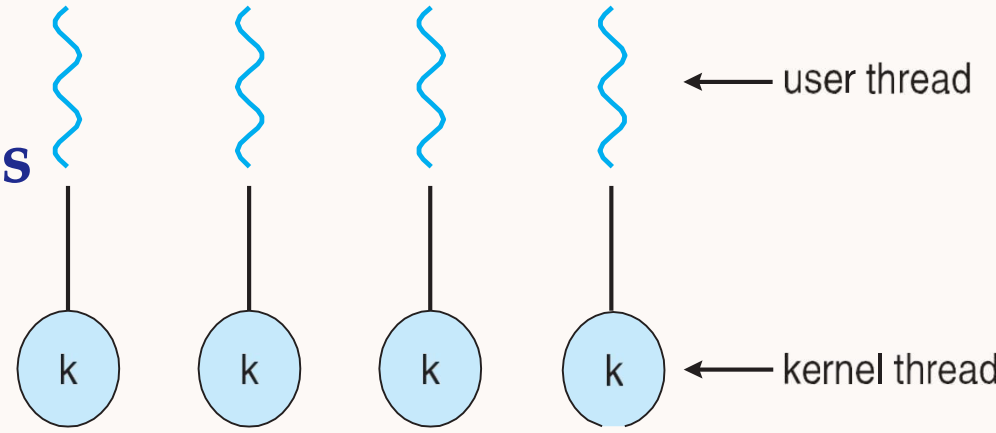
- Many user-level threads mapped to single kernel thread
- One thread blocking causes all to block
- Multiple threads may not run in parallel on muti-core system because only one may be in kernel at a time



spot_to_be_comp

ONE-TO-ONE

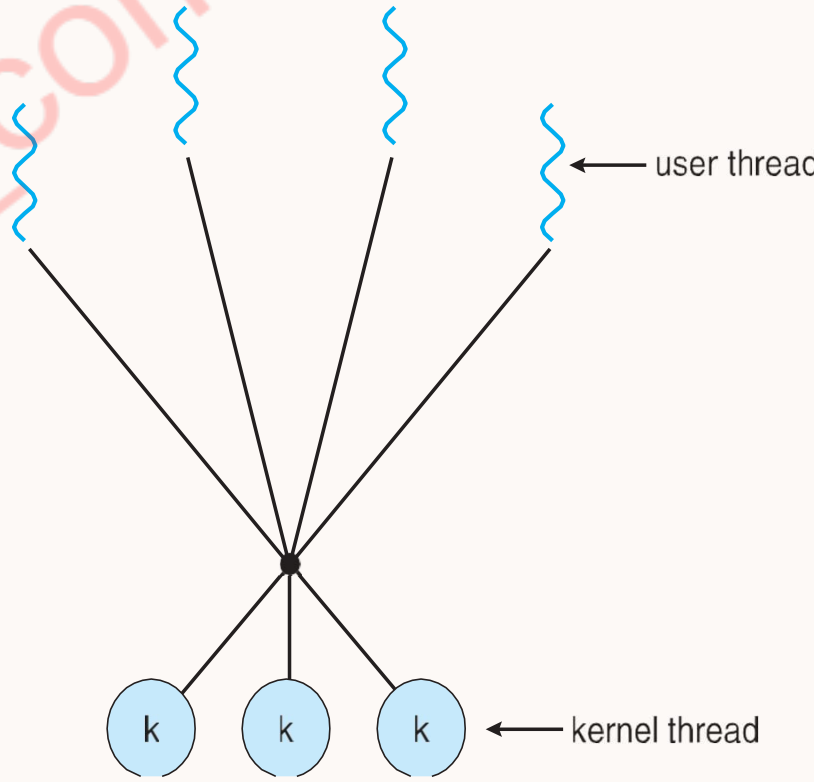
- Each user-level thread maps to kernel thread
- Creating a user-level thread creates a kernel thread
- More concurrency than many-to-one
- Number of threads per process sometimes restricted due to overhead



spu_te_be_comp

MANY-TO-MANY MODEL

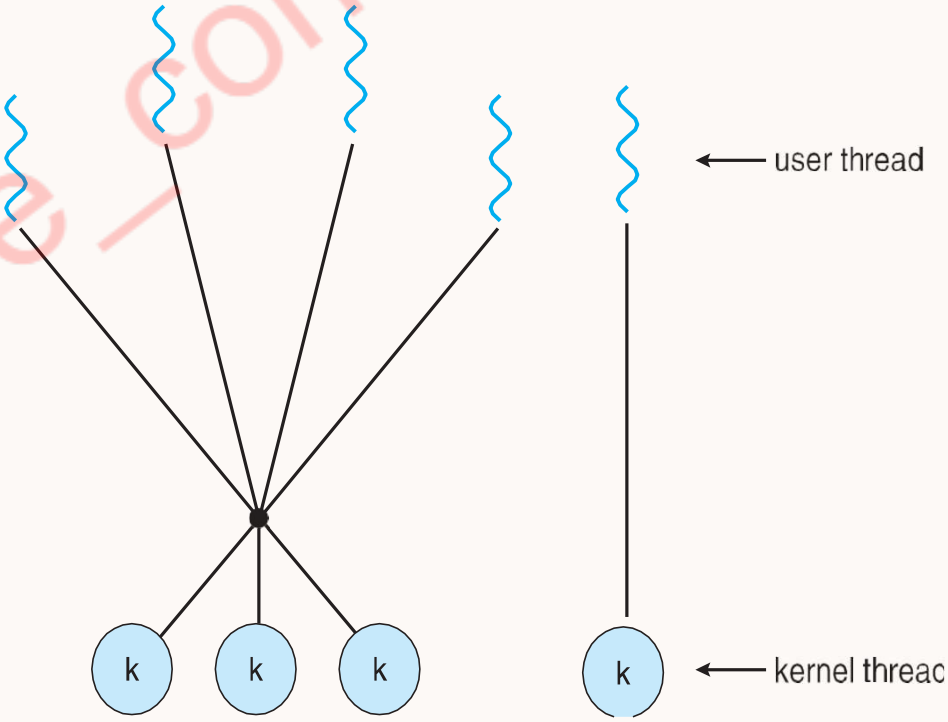
- Allows many user level threads to be mapped to many kernel threads
- Allows the operating system to create a sufficient number of kernel threads



sppu_te_be_comp

TWO-LEVEL MODEL

- Variation of many-to-many model.
- Multiplexes many user level threads to a smaller or equal number of kernel threads.
- Also allows a user-level thread to be bound to a kernel thread.



sppu_to_be_comp

THREAD MANAGEMENT FUNCTIONS IN C

- In C language, POSIX **<pthread.h>** standard API (Application program Interface) for all thread related functions.
- To execute the C programs that uses these functions, we may have to use the -pthread or -lpthread in the command line while compiling the file.

```
gcc -pthread file.c
gcc -lpthread file.c
```

spcu_te_be_comp

THREAD CREATION

- To create a new thread using Pthreads in C or C++, the pthread_create() function is used.
- Syntax :

```
#include <pthread.h>

int pthread_create(pthread_t *thread,
                   const pthread_attr_t *attr,
                   void *(*start_routine)(void *),
                   void *arg);
```

sppu_te_be_comp

- **Where,**
  - **pthread_t *thread:** A pointer to a pthread_t variable where the system will store the ID of the newly created thread.
  - **const pthread_attr_t *attr:** A pointer to a thread attributes object. This allows you to specify various properties of the new thread (e.g., stack size, scheduling policy). Passing NULL uses default attributes.
  - **void *(*start_routine)(void *):** A pointer to the function that the new thread will execute.
  - **void *arg:** A single argument passed to the start_routine function. If the function requires multiple arguments, they can be encapsulated within a structure and a pointer to that structure can be passed. If no arguments are needed, NULL can be passed.

spit_it_bf_comp

```c
#include <stdio.h>
#include <pthread.h>

int a = 10, b = 5; // Example values

void* add(void* arg) {
    printf("Addition: %d + %d = %d\n", a, b, a + b);
    return NULL;
}

void* subtract(void* arg) {
    printf("Subtraction: %d - %d = %d\n", a, b, a - b);
    return NULL;
}

int main() {
    pthread_t t1, t2;

    pthread_create(&t1, NULL, add, NULL);
    pthread_create(&t2, NULL, subtract, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    return 0;
}
```

spl_te_be_comp

Return Value:

- Returns 0 on success and returns an error number on failure.

sppu_te_be_comp

THREAD COMPLETION

1. Waiting for Thread Completion (Joining Threads):

Syntax:

```
#include <pthread.h>

int pthread_join(pthread_t thread, void **retval);
```

Parameters:

- **thread:** pthread_t identifier.
- **retval:** pointer to a void* pointer. If the target thread returns a value using pthread_exit(), that value will be stored at the location pointed to by *retval. If you don't need the return value, you can pass NULL.

scnu_te_be_comp

2. Terminating a Thread:

Syntax:

```
#include <pthread.h>

void pthread_exit(void *retval);
```

Parameters:

- **retval:** This argument is a pointer to a value that the thread wants to return upon its termination.

sppu_te_be_comp

3. Detaching a Thread: meaning it will automatically release its resources upon termination.

Syntax:

```
#include <pthread.h>

int pthread_detach(pthread_t thread);
```

Parameters:

- **thread:** This is the pthread_t identifier of the thread to be detached.

spu_te_be_comp

4. Implicit Termination:

- A thread also implicitly terminates when its entry function (the function passed to pthread_create()) returns.
- The return value of this function will be passed to pthread_join() if a thread joins it.

sppu_te_be_comp

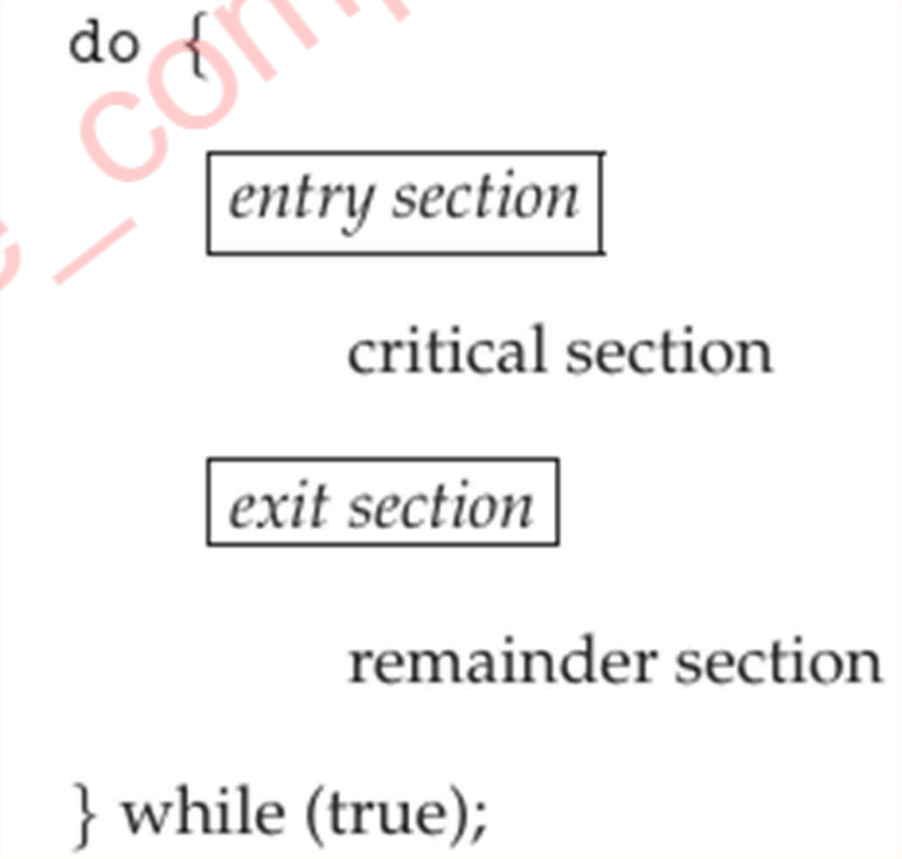
CRITICAL SECTION

- When a process executes code that manipulates shared data (or resource), we say that the process is in it's critical section (CS) (for that shared data)
- The execution of critical sections must be mutually exclusive: at any time, only one process is allowed to execute in its critical section (even with multiple CPUs)
- Then each process must request the permission to enter it's critical section (CS)

sonu_te_be_comp

CRITICAL SECTION CONTD...

- **Each process must request permission to enter its critical section.**
  - The section of code implementing this request is the **entry section.**
  - The critical section may be followed by an **exit section.**
  - The remaining code is the **remainder section.**



scanned_by_be_comp

SOLUTION TO THE CRITICAL-SECTION PROBLEM

1. **Mutual Exclusion –** only 1 process will execute in critical section.
2. **Progress -** processes that are not executing in their remainder sections can participate in deciding which will enter its critical section next.
3. **Bounded Waiting -** exists a bound, or limit, on the number of times that other processes are allowed to enter CS and before that request is granted.

sppu_te_be_comp

WHAT ABOUT PROCESS FAILURES?

- **If all 3 criteria (ME, progress, bounded waiting) are satisfied, then a valid solution will provide robustness against failure of a process in its remainder section (RS)**
  - Since failure in RS is just like having an infinitely long RS
- **However, no valid solution can provide robustness against a process failing in its critical section (CS)**
  - A process Pi that fails in its CS does not signal that fact to other processes: for them Pi is still in its CS

sonu-te_be_comp

MUTUAL EXCLUSION

- A property of process synchronization that states that

"no two processes can exist in the critical section at any given point of time"

- Race condition can be avoided due to mutual exclusion.
- Mutual exclusion works only for shared data.

spu_te_be_comp

REQUIREMENTS FOR MUTUAL EXCLUSION

1. **Mutual exclusion must be enforced**
2. **A process in its noncritical section must not affect others.**
3. **No process should face indefinite delay – avoid deadlock and starvation.**
4. **If no one is in the critical section, entry must be granted immediately to the process who request.**
5. **No assumptions about process speed or processor count.**

spot_te_be_comp

MUTUAL EXCLUSION: HARDWARE SUPPORT

1. **Interrupt Disabling:** To guarantee mutual exclusion, it is sufficient to prevent a process from being interrupted.

A process can then enforce mutual exclusion in the following way:

```
while (true) {
        /* disable interrupts */;
        /* critical section */;
        /* enable interrupts */;
        /* remainder */;
}
```

sppu_te_be_comp

LOCKS

- **Another important functions provided by POSIX library files is of Lock to ensure mutual exclusion to a critical section.**
- **As an example, assume our critical section looks like this, the canonical update of a shared variable:**

```
balance = balance + 1;
```

spu_te_be_comp

- To use a lock, we add some code around the critical section like this:

```
lock_t mutex; // some globally-allocated lock 'mutex'
...
lock(&mutex);
balance=balance+ 1;
unlock(&mutex);
```

- A lock is just a variable, and thus to use one, you must declare a lock variable of some kind (such as mutex above)

sppu_te_be_comp

- **Controlling Interrupts :** One of the earliest solutions used to provide mutual exclusion was to disable interrupts for critical sections; this solution was invented for single-processor systems.

```
void lock( )
{
        DisableInterrupts( );
}
void unlock( )
{
        EnableInterrupts( );
}
```

sppu_te_be_comp

SEMAPHORE

- **Mutex locks, are generally considered the simplest of synchronization tools.**
- **Another approach similar to mutex lock which can provide more sophisticated ways for processes to synchronize their activities are Semaphores.**
- **A semaphore S is –**
  - an integer variable that is used to control access to a shared resource and
  - accessed only through two standard atomic operations: wait() and signal().

sppu_te_be_comp

1. The wait() operation was originally termed P;

- Decreases the semaphore value.
- If the value becomes negative, the process is blocked until the resource becomes available.

```
wait(S) {
    while (S <= 0)
        ; // busy wait
    S--;
}
```

sppu_te_be_comp

2. **The signal() was originally called V. The definition of wait() is as follows:**

- **Increases the semaphore value.**
- **If there are waiting processes, one of them gets unblocked.**

```
signal(S) {
    S++;
}
```

- **Note:-** All modifications to the integer value of the semaphore in the wait() and signal() operations must be executed indivisibly.

sppu_te_be_comp

Why Semaphores are Used

- **Synchronization:** They coordinate the actions of multiple processes to ensure they execute in a proper sequence.
- **Mutual Exclusion:** Binary semaphores prevent more than one process from entering a critical section simultaneously, avoiding corrupted data.
- **Resource Management:** Counting semaphores manage access to a finite set of resources, ensuring that the number of users does not exceed the available count.
- **Prevent Race Conditions:** By controlling access, semaphores stop race conditions where multiple processes compete for a resource, leading to unpredictable outcomes.

spit_te_be_comp

Types of Semaphore

- **Binary Semaphore:** A semaphore whose value can only be 0 or 1. It functions like a mutex (mutual exclusion lock), ensuring that only one process can access a critical section at a time.
- **Counting Semaphore:** A semaphore that can take on any non-negative integer value. It's used to control access to a resource that has multiple instances, such as a printer or a pool of memory.

sppu_te_be_comp

- **Semaphore initialization:**

```
#include <semaphore.h>
sem_t s;
sem_init (&s, 0, 1);
```

- **Where,**
  - 1st argument is semaphore name, for this declare semaphore variable with sem_t data type.
  - 2nd argument is will be set to 0 in all of the examples because it indicates that the semaphore is shared between threads in the same process.
  - 3rd argument is initial value initialize to the semaphore.

spou_te_be_comp

- **A semaphore can be manipulated with two routines only; in the POSIX standard, these routines are sem_wait() and sem_post().**

```
wait(semaphore *S) {
        S->value--;
        if (S->value < 0) {
                add this process to S->list;
                block();
        }
}
```

```
signal(semaphore *S) {
        S->value++;
        if (S->value <= 0) {
                remove a process P from S->list;
                wakeup(P);
        }
}
```

sppu_te_be_comp

CLASSICAL PROBLEMS OF SYNCHRONIZATION

- **Classical problems used to test newly-proposed synchronization schemes**
  1. **Bounded-Buffer Problem**
  2. **Readers and Writers Problem**
  3. **Dining-Philosophers Problem**
- Demonstrate challenges that arise when multiple processes or threads need to access shared resources, leading to potential issues like data corruption, deadlock, or starvation if not properly synchronized.

sppu_te_be_comp

BOUNDED-BUFFER PROBLEM

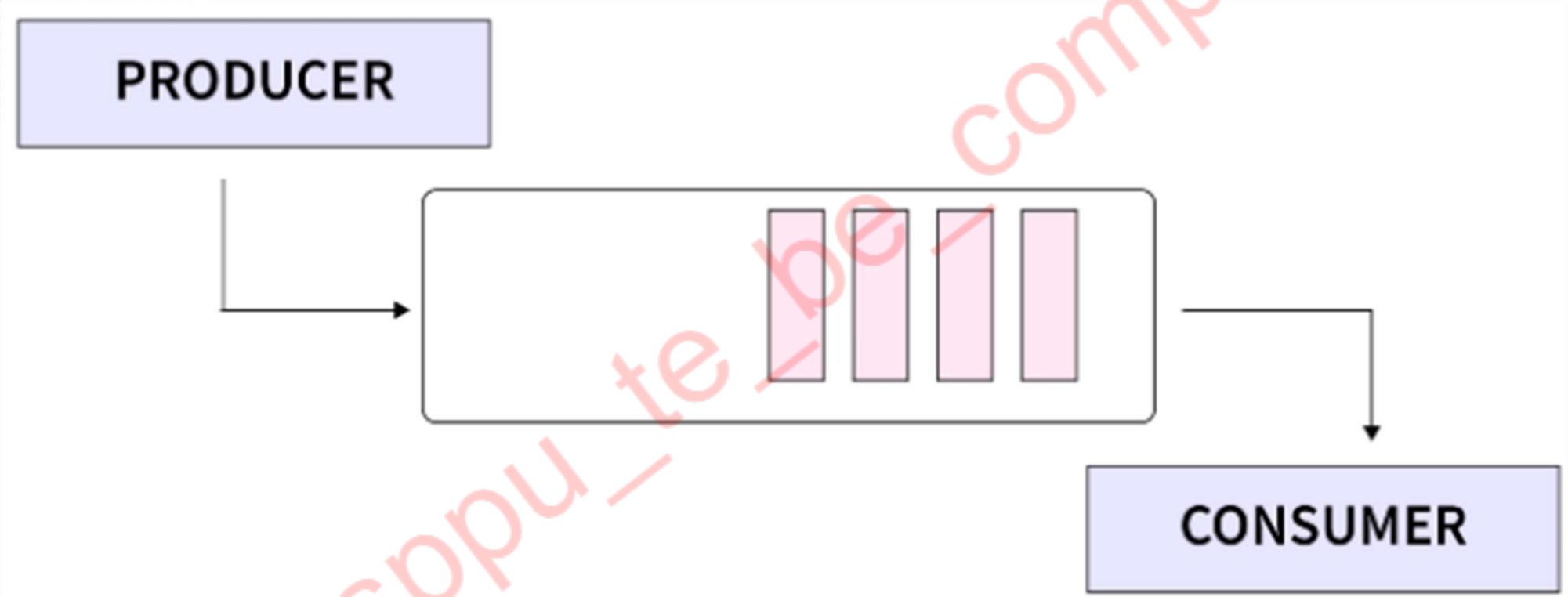
- **Also known as Producer/Consumer Problem.**
- **With the presence of more than one process and limited resources in the system the synchronization problem arises.**
- **If one resource is shared between more than one process at the same time then it can lead to data inconsistency.**

sppu_te_be_comp

WHAT IS PRODUCER CONSUMER PROBLEM?

- In operating System Producer is a process which is able to produce data/item.
- Consumer is a Process that is able to consume the data/item produced by the Producer.
- Both Producer and Consumer share a common memory buffer. This buffer is a space of a certain size in the memory of the system which is used for storage. The producer produces the data into the buffer and the consumer consumes the data from the buffer.

sppu_te_be_comp



sppu_te_be_comp

- So, what are the Producer-Consumer Problems?

1. Producer Process should not produce any data when the shared buffer is full.
2. Consumer Process should not consume any data when the shared buffer is empty.
3. The access to the shared buffer should be mutually exclusive i.e at a time only one process should be able to access the shared buffer and make changes to it.

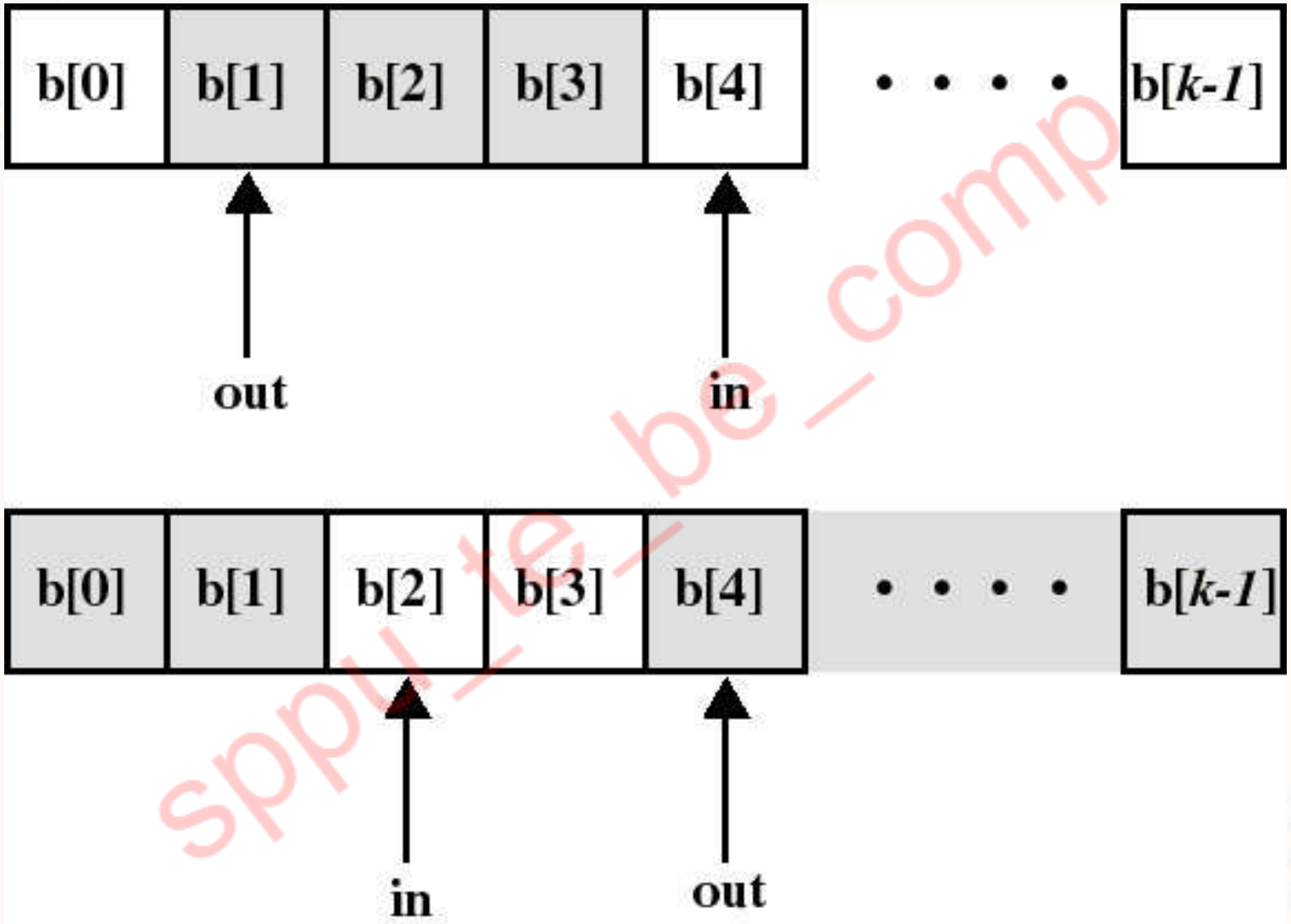
sppu_te_be_comp

SOLUTION FOR PRODUCER CONSUMER PROBLEM

- **To solve the Producer-Consumer problem three semaphores variable are used :**

1. **Full:** Track filled spaces in buffer and initialized to 0.
2. **Empty:** Track empty space in buffer and initialized to buffer-size.
3. **Mutex:** Used to achieve mutual exclusion. Initialized as binary semaphore with value 1.
4. In and out to variables. **`in`** points to the next item to be produced and **`out`** points to the next item to be consumed

spou_te_be_comp



sppu_te_be_comp

BOUNDED-BUFFER PROBLEM

- *n* buffers, each can hold one item
- Semaphore **`mutex`** initialized to the value 1
- Semaphore **`full`** initialized to the value 0
- Semaphore **`empty`** initialized to the value n

sppu_te_be_comp

BOUNDED BUFFER PROBLEM (CONT.)

- The structure of the producer process

```
do {
        ...
        /* produce an item in next_produced */
        ...
     wait(empty);
     wait(mutex);
        ...
        /* add next produced to the buffer */
        ...
     signal(mutex);
     signal(full);
} while (true);
```

sppu_te_be_comp

BOUNDED BUFFER PROBLEM (CONT.)

- The structure of the consumer process

```
Do {
    wait(full);
    wait(mutex);
        ...
    /* remove an item from buffer to next_consumed */
        ...
    signal(mutex);
    signal(empty);
        ...
    /* consume the item in next consumed */
        ...
 } while (true);
```

sppu_te_be_comp

READERS-WRITERS PROBLEM

- This involves coordinating access to a shared resource by multiple threads, some of which only **read (readers)** and others which **modify (writers)** the data.
- **A data set is shared among a number of concurrent processes**
- **Readers –** only read the data set; they do not perform any updates
- **Writers –** can both read and write

sppu_te_be_comp

- **The core rules are:**
  - Multiple readers can access the shared resource concurrently.
  - Only one writer can access the shared resource at a time.
  - No reader can access the resource while a writer is accessing it.
  - No writer can access the resource while a reader is accessing it.

- **Problem –**
  - Allow multiple readers to read at the same time
  - Only one single writer can access the shared data at the same time

sppu_te_be_comp

- **Shared Data**
  - Data set
  - Semaphore **rw_mutex** common to both reader & writer process. Initialized to 1
  - Semaphore **mutex** used to ensure mutual exclusion. Initialized to 1
  - Integer **read_count** variable keeps track of how many processes are currently reading the object. Initialized to **0**
- **Note-** Semaphore **rw_mutex** is used by writers and 1st or last reader. It will not be used by readers who enter or exit while other readers are in CS.

scanute_be_comp

- The structure of a writer process

```
do {
    wait(rw_mutex);
        ...
    /* writing is performed */
        ...
    signal(rw_mutex);
} while (true);
```

sppu_to_be_comp

- The structure of a reader process

```
do {
   wait(mutex);
   read_count++;
   if (read_count == 1)
      wait(rw_mutex);
   signal(mutex);
      . . .
   /* reading is performed */
      . . .
   wait(mutex);
   read_count--;
   if (read_count == 0)
      signal(rw_mutex);
   signal(mutex);
} while (true);
```

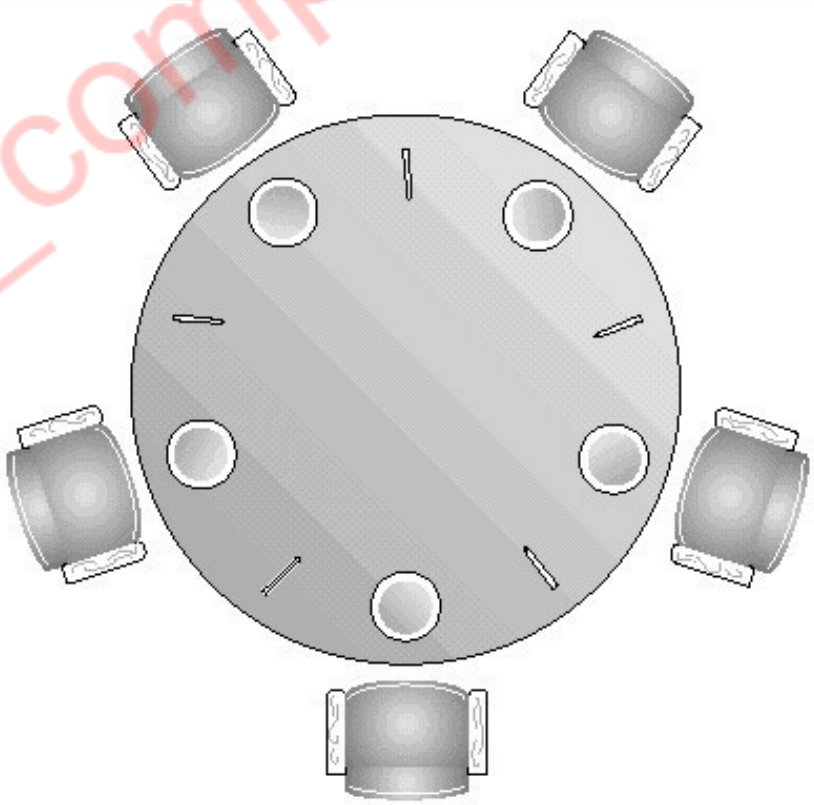
spu_te_be_comp

THE DINING-PHILOSOPHERS PROBLEM

- The Dining Philosophers Problem is a classic concurrency problem illustrating synchronization challenges, particularly deadlock and starvation, when multiple threads compete for shared resources.
- 5 philosophers who only eat and think.
- Each need to use 2 forks for eating. Total available quantity of forks are 5.

sppu_te_be_comp

- The core rules:
  - Philosophers: Represented by threads. Each philosopher alternates between thinking and eating.
  - Forks: Represented by mutexes. There's one fork between each adjacent pair of philosophers.
  - Eating: To eat, a philosopher needs to acquire both the left and right forks.



spcu_te_be_comp

THE PROBLEM SETUP

- **Philosophers:** Multiple processes or entities that need to access shared resources.
- **Chopsticks/Forks:** Limited, exclusive resources (like a database lock or a printer).
- **Circular Table:** Implies a circular dependency, as each philosopher has a left and right chopstick.
- **States:** Each philosopher alternates between:
  - **Thinking:** No resources are in use.
  - **Hungry:** Wants to eat and is trying to acquire chopsticks.
  - **Eating:** Both left and right chopsticks are held and in use.

scpu_te_be_comp

- **The Synchronization Challenge**
  - **Deadlock:** A situation where all philosophers pick up their left (or right) chopstick simultaneously and become unable to pick up their right (or left) chopstick, creating a circular waiting condition where none can proceed.
  - **Starvation:** A philosopher repeatedly tries to acquire resources but never succeeds, even if other processes are able to.

spcl_fe_be_comp

Solutions using Semaphores:

- A binary semaphore can represent each chopstick, initialized to 1.
- A philosopher performs a wait() (or P()) operation to acquire a chopstick (decrementing the semaphore) and a signal() (or V()) to release it.
- To prevent deadlock, solutions include:
  - Limiting concurrent philosophers: Allow only N-1 philosophers to attempt to pick up chopsticks at any time.
  - Asymmetrical approach: Have some philosophers pick up their right chopstick first, then left, while others do the opposite.

sppu_te_be_comp

Why It Matters

- The Dining Philosophers Problem highlights how complex it is to design effective resource allocation algorithms.
- It demonstrates that a simple, naive approach to resource acquisition (like each philosopher picking up their left chopstick first) can lead to disastrous concurrency issues like deadlocks and starvation.
- Solutions to this problem provide foundational techniques for ensuring correct and fair resource sharing in operating systems and other concurrent systems, such as multi-threaded applications or database transactions.

spu_te_he_comp

THE DINING PHILOSOPHERS PROBLEM

- Each philosopher is a process
- One semaphore per fork:
  - fork: array[0..4] of semaphores
  - Initialization: fork[i].count:=1 for i:=0..4
- A first attempt:
- Deadlock if each philosopher start by picking his left fork!

```
Process Pi:
repeat
 think;
 wait(fork[i]);
 wait(fork[i+1 mod
5]);
 eat;
 signal(fork[i+1 mod
5]);
 signal(fork[i]);
forever
```

spou_te_be_comp

THE DINING PHILOSOPHERS PROBLEM

- A solution: admit only 4 philosophers at a time that tries to eat
- Then 1 philosopher can always eat when the other 3 are holding 1 fork
- Hence, we can use another semaphore T that would limit at 4 the number of philosophers "sitting at the table"
- Initialize: T.count:=4

```
Process Pi:
repeat
 think;
 wait(T);
 wait(fork[i]);
 wait(fork[i+1 mod 5]);
 eat;
 signal(fork[i+1 mod 5]);
 signal(fork[i]);
 signal(T);
forever
```

sapute_bs_comp

COMMON CONCURRENCY PROBLEM

- **Common Concurrency problems are also known as concurrency bugs.**
- **Concurrency bugs are defects that occur when the timing or order of multiple threads or processes interacting with shared resources leads to incorrect behaviour.**
- **They are broadly categorized into two:**
  - **Deadlock bugs, and**
  - **Non-deadlock bugs**

sppu-te-be-comp

NON DEADLOCK BUGS

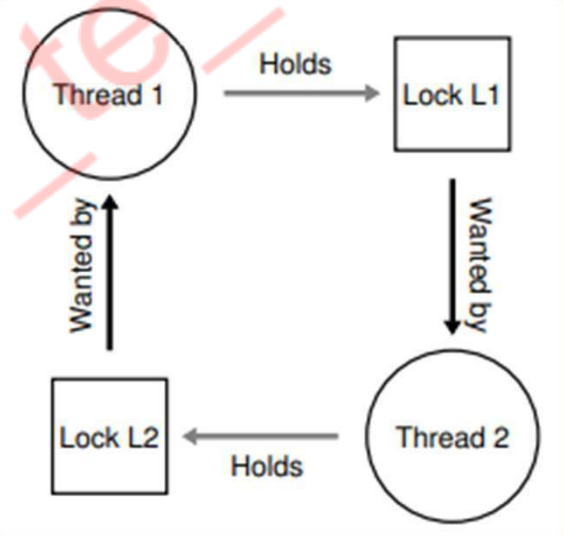
1. **Atomicity Violations:** Happens when a sequence of operations that should be treated as a single, indivisible (atomic) unit is interrupted by another thread.

2. **Order Violations:** Occur when the expected sequence of operations between threads is not maintained, leading to incorrect logic.

sppu_te_be_comp

DEADLOCK BUGS

- A deadlock in an OS occurs when two or more processes are indefinitely blocked, each waiting for a resource held by another process in the same set, creating a circular dependency that prevents any process from completing its execution.



sppu_te_be_comp

- **In deadlock, process never complete its execution & system resources are held up.**
- **Necessary conditions for deadlock-**
  - **Mutual exclusion -** At least one resource must be held in a non sharable mode.
  - **Hold and wait**
  - **No preemption -** Resources cannot be preempted
  - **Circular wait**

sppu_te_be_comp

METHODS FOR HANDLING DEADLOCKS

3 ways to mange deadlock-

- First, ensure system never enter a deadlock, use a protocol to prevent or avoid deadlocks.
- Second, let the system to enter a deadlock state, detect it & recover from it.
- Third, ignore the issue & imagine deadlock never happen in the system.

spnu_te_be_comp

DEADLOCK PREVENTION

- Deadlock prevention algorithms guarantee that one of the necessary conditions does not hold.
- A system that violates one of the four conditions required for the event of deadlock.
- This guarantees the system stays free from deadlock.

sppu_te_be_comp

DEADLOCK PREVENTION CONTINUED…

Mutual Exclusion

- This condition must hold for non-sharable resources.
- Sharable resources don't require mutually exclusive access.

Hold & Wait

- Processes demand all resources at same time.
- Before requesting new resource previous resource will be discharge.

sppu_te_be_comp

DEADLOCK PREVENTION CONTINUED…

- **No Preemption**
  - A process cannot hold old resource while requesting for new resource.
  - When a resource is requested & not accessible, then system finds which processes have them.
- **Circular Wait**
  - To avoid circular wait, number all resources & process request resource in increasing order.
  - To demand resource, process sould discharge all old resource.

sppu_to_be_comp

DEADLOCK AVOIDANCE

- Necessitates that the system has some extra from the priori data available.
- Each process declare the greatest number of resources of each kind that it might require.
- The algorithm inspects the resource-allocation state to guarantee that there can never be a circular-wait condition.

sppu_te_be_comp

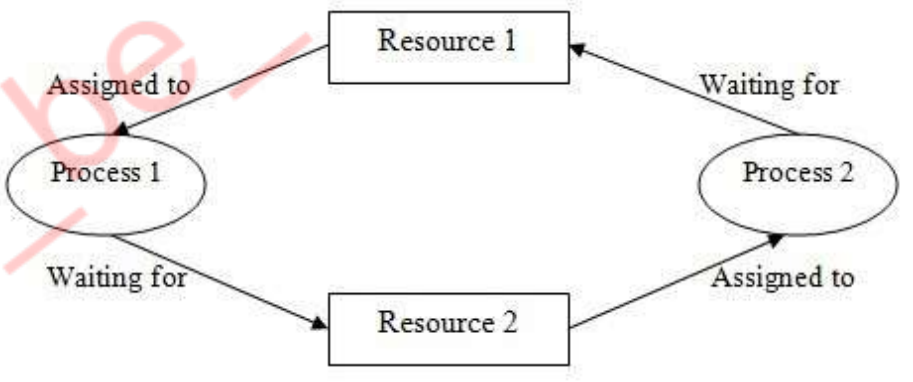
DEADLOCK DETECTION

- Method for verifying that a deadlock exists & distinguishing processes & resources engaged in the deadlocked.
- The main idea is to check resource available against allocated for safe state.
- When a deadlock is recognized, there should be few choices:
  - Single instance of resource
  - Multiple instance of resource

sppu_te_be_comp

DEADLOCK DETECTION CONTINUED…

- Single instance of resource
  - Detection of cycle in RAG represent presence of deadlock.



- Multiple instance of resource
  - Detection of cycle is not sufficient condition here, the system might halt in case of deadlock.

sppu_te_be_comp

RECOVERY FROM DEADLOCK

- 2 methodologies of breaking a deadlock:

1. **Process Termination:** kill at least one process.
   - Abort all the deadlocked processes
   - Abort one process in turn until deadlock is removed.
2. **Resource Preemption:** preempt few resources.
   - Choosing a victim
   - Rollback
   - Starvation

sppu_te_be_comp

THANK YOU

sppu_te_be_comp